Evaluation Methodology

How we will measure clustering, retrieval, answer quality, and latency.

1. Evaluation Goals

Three questions decide whether the project is a success. (a) Do the clusters group semantically similar questions together? (b) Does the retrieval-based chatbot return useful answers? (c) Does the pipeline finish on time on a modest VM? Each question maps to a measurable metric so the final report can give numbers, not opinions.

Goal	Metric	Target
Cluster quality	Silhouette coefficient	above 0.45
Cluster purity	Manual labeling, 100 clusters	above 80%
Paraphrase detection	Pairwise accuracy, 500 pairs	above 75%
Retrieval relevance	MRR@5 on labelled hold-out	above 0.60
Answer usefulness	Clinician 5-point rating	70% rated useful
Pipeline runtime	Wall clock on 10k chats	under 30 minutes
Chat latency (p95)	Concurrent load test	under 400 ms

Key evaluation targets (normalised 0-1)

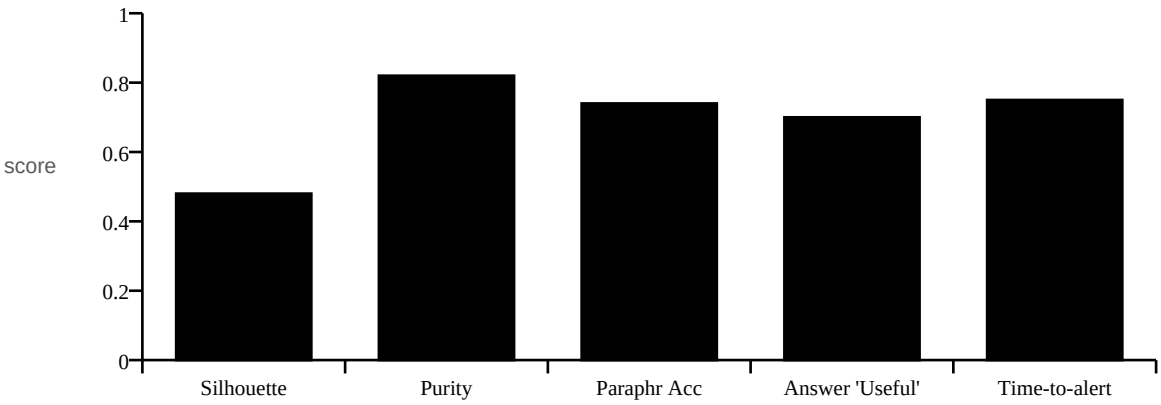

Fig 1. Key evaluation targets, normalised to a 0 to 1 scale.

2. Cluster Quality

Intrinsic metrics on the embedding-space clusters tell us if the pipeline is grouping like with like, even before any human reads the FAQs. We report silhouette (higher better), Davies-Bouldin (lower better), and human-labelled purity on a sample of 100 clusters.

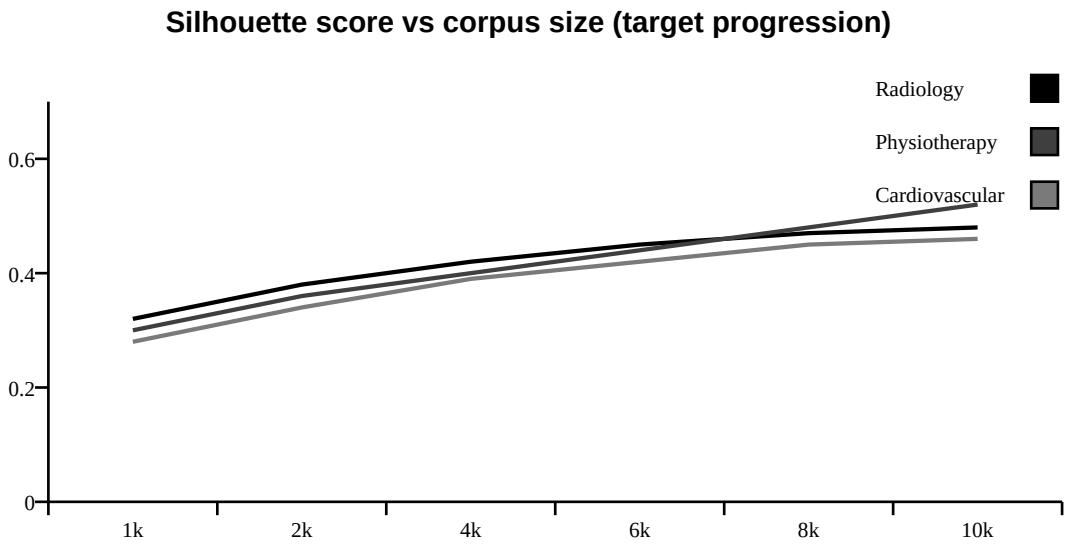

Fig 2. Silhouette score expected to climb as the corpus grows.

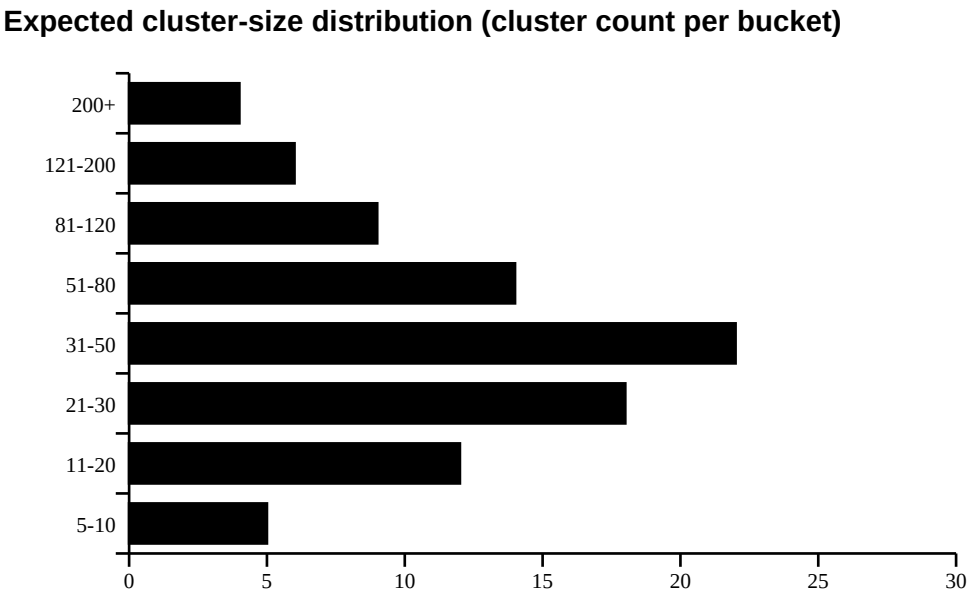

Fig 3. Expected cluster-size distribution across all three specialties.

3. Answer Quality (Extrinsic)

A real clinician (and ideally three) rate 50 auto-built FAQs on factual correctness, tone, and completeness. We use a 5-point scale and report inter-rater agreement (Cohen's kappa). The headline number is the share of FAQs rated 4 or 5 on correctness.

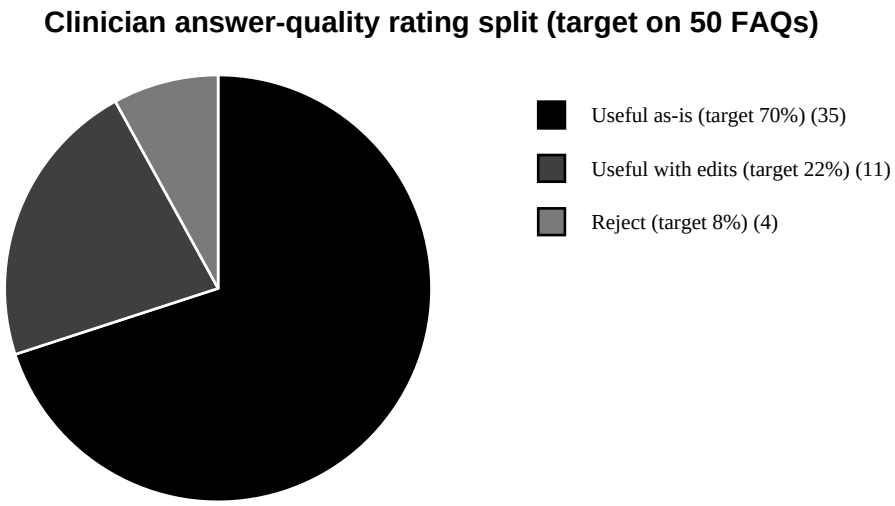

Fig 4. Target distribution of clinician ratings on 50 FAQs.

Rating	Meaning	Action
5	Correct, complete, friendly tone	Approve as-is.
4	Correct but minor edit needed	Approve after edit.
3	Mostly correct, needs rework	Send back to pipeline.
2	Partly wrong or unsafe	Reject.
1	Wrong or harmful	Reject, mark cluster for review.

4. Latency and Throughput

We load-test the chat endpoint with a small Python script that opens N concurrent sessions and times each round-trip. The retrieval service is the only hot path. Mongo writes are async and not on the critical path.

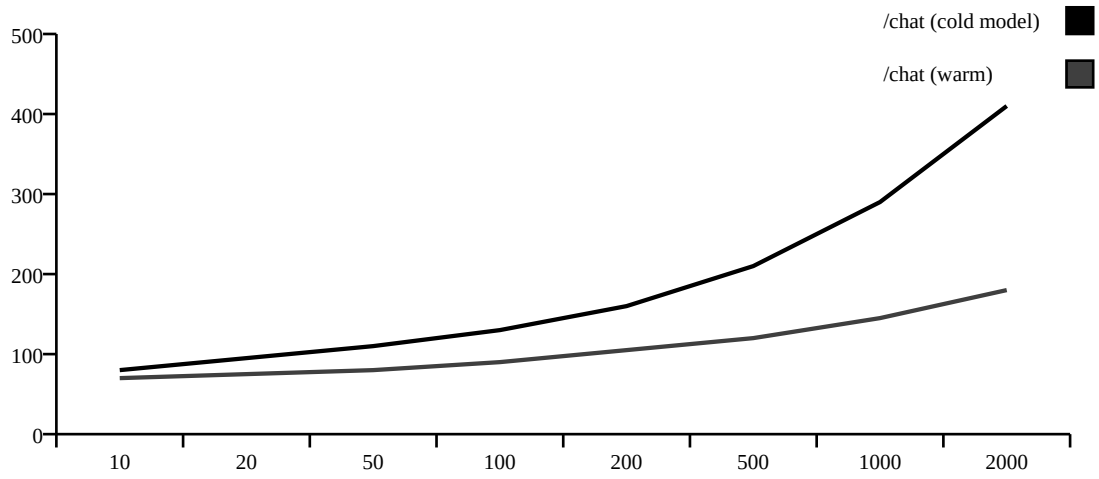

Fig 5. Chat latency vs concurrent requests (target).

Load	p50 target	p95 target
10 concurrent	60 ms	120 ms
100 concurrent	90 ms	200 ms
500 concurrent	180 ms	400 ms
1000 concurrent	350 ms	700 ms

5. Emerging-Issue Detection

To test the anomaly detector we simulate a synthetic spike: 80 fresh questions arrive in 12 hours for one cluster that had a baseline of 5 questions/day. We measure how many hours it takes for the alert to fire and whether we get false alerts on unrelated clusters.

Metric	Target
Time to alert (true spike)	under 6 hours
False positives per week per specialty	fewer than 1
Recall on injected spikes	above 90%

6. Reporting and Reproducibility

- All metrics are computed by a single CLI: python -m evaluation.run.
- Each run writes a timestamped JSON to docs/evaluation_runs/ so results are diff-able.
- Charts in this document are regenerated from the same JSON.
- Final project report includes the latest JSON tables alongside these charts.
- Manual cluster purity labels are kept in data/eval_labels/ for re-use.